

Accurate Specification Recovery for Effective Test Generation

Junda Zhao 

University of Toronto
Department of Mechanical & Industrial Engineering
Toronto, Canada
junda.zhao@mail.utoronto.ca

Abstract

Large language model (LLM)-based unit-test generation has largely been studied under the assumption that the code under test is correct, using coverage or test-correctness metrics that do not directly measure defect detection. When the code under test is buggy, however, models can be guided toward assertions that encode the faulty behavior, producing tests that confirm rather than reveal defects. Our recent work quantified this *misguidance effect* and showed that a specification-based approach, which first recovers a behavioral specification of the focal method from its buggy implementation and then generates tests from that specification, can substantially mitigate it. Although this establishes a specification-recovery-first paradigm for LLM-based unit-test generation, the recovery step itself remains simple and its specifications are themselves fallible. This dissertation therefore aims to make specification recovery from imperfect code accurate, evidence-driven, and uncertainty-aware, so that it consistently yields effective bug-finding tests.

CCS Concepts: • Software and its engineering → Software testing and debugging; Automatic programming.

Keywords: Unit Test Generation, Large Language Models, Specification Recovery, Bug Detection

ACM Reference Format:

Junda Zhao. 2026. Accurate Specification Recovery for Effective Test Generation. In *Companion Proceedings of the 2026 ACM SIGPLAN International Conference on Systems, Programming, Languages, and Applications: Software for Humanity (SPLASH Companion '26)*, October 4–9, 2026, Oakland, CA, USA. ACM, New York, NY, USA, 3 pages. <https://doi.org/10.1145/3837729.3842146>

1 Motivation

Unit testing is a cornerstone of software quality assurance, valued primarily for early defect detection [20]. Recent work has increasingly automated per-method unit-test generation

with Large Language Models (LLMs) [3, 22], but these studies are predominantly evaluated on code assumed correct, using coverage and test-correctness metrics that correlate only weakly with actual fault-detection power [12]. In practice, test generation is routinely applied precisely when correctness is uncertain, such as when reproducing reported bugs, reviewing changes, or following test-driven workflows in which the implementation is not yet correct [2]; accordingly, studies that feed buggy code to LLM-based generators report markedly degraded bug detection [1, 11]. A test that merely confirms buggy behavior certifies the defect and prevents its detection [8, 15] (Figure 1).

Prior work recognizes this problem but does not resolve it. Huang et al. quantify how buggy code misguides generation, but their metric counts every test that passes on the buggy version as misguided, over-counting tests that agree with correct behavior [11]; the hazard is also documented for pre-LLM search-based generators [18]. Documentation- and oracle-generation methods presume trustworthy documentation or learn from the code itself [9, 10, 22], and intent inference from buggy code has been validated only on small programs [14]. Classical specification mining infers likely invariants from executions of the program or its tests [5, 16], so mined specifications reflect observed behavior, including a defect’s; symbolic, search-based, and model-checking generators likewise treat the code as ground truth [6, 17]. None recovers the *intended* behavior of imperfect code, the gap this dissertation addresses.

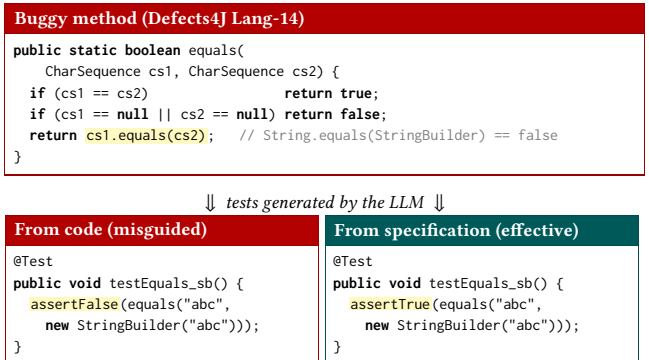


Figure 1. Tests from buggy `equals` assert the bug; tests from its recovered specification, “returns true iff both sequences have the same characters, regardless of type,” expose it.



This work is licensed under a Creative Commons Attribution 4.0 International License.

SPLASH Companion '26, Oakland, CA, USA

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2910-2/2026/10

<https://doi.org/10.1145/3837729.3842146>

2 Problem

Our recent study established and quantified the *misguidance effect*: prompting an LLM with buggy code steers it to treat the erroneous behavior as intended, increasing *misguided tests*, that is, tests that consistently pass on the buggy version but fail on the fixed one (flaky tests are excluded by repeated execution), while suppressing *effective tests* that detect the bug [23]. The effect is severe across eleven state-of-the-art models and visible in their internal token preferences. To mitigate it, we drew on specification-based testing [7] and proposed replacing the code in the prompt with an LLM-recovered *behavioral specification* of the focal method: a docstring-style description of its intended inputs, outputs, and exceptional behavior (e.g., the specification in Figure 1); the buggy implementation is then withheld from the test-generation prompt. Tests generated from this specification show markedly fewer misguided tests and higher bug detection on buggy inputs, without degrading tests for bug-free code. Because whether the code under test is buggy or bug-free is unknown at generation time, the paradigm must recover the specification from the code as is; specifications recovered from bug-free code serve only as an upper bound. Manual inspection showed this is viable: the defect is substantially blocked from propagating into the specification and the intended behavior is recovered for a large fraction of buggy methods, and it is this recovered behavior, not merely withholding the code, that yields effective tests [23].

While this validates a specification-recovery-first paradigm, its design space remains largely unexplored: recovery is a single prompt over the focal method and its in-class context, and misguided tests remain. Crucially, a recovered specification is itself fallible. Substituting a fallible specification for the code would merely move the assumption of correctness from one to the other. This dissertation instead treats a recovered specification as a set of behavioral claims to be validated against evidence and annotated with uncertainty, and asks which evidence and recovery workflows yield high-quality specifications, which representation best supports test generation, and how these combine end to end.

3 Approach

Buggy code is useful but potentially misleading evidence about intended behavior; recovery should therefore identify which behavioral claims are supported, which may be inherited from the defect, and which remain uncertain, rather than turn an unsupported assumption into a test oracle.

The first direction studies what evidence is useful for recovering intended behavior from imperfect code. Our framework organizes evidence into tiers from the focal method outward (class surface, type surface, call graph, usage examples, repository-level context) and ablates them cumulatively, under a guard that excludes context selected through the focal body, which could re-introduce the bug. More context is not

always better: it may be irrelevant, stale, or affected by the same defect, and documentation conventions differ across projects (e.g., user- versus developer-facing). We will evaluate which tiers improve specifications and bug detection, when they add noise, and whether repository characteristics explain non-uniform effects.

The second direction studies how recovered specifications should be represented. Docstrings may leave behavior implicit or ambiguous. We will compare it with more structured forms, including requirement lists, contracts (preconditions, postconditions, exceptional behavior), checkable properties, input-output examples, and explicit fault hypotheses [9, 10], to identify representations that guide LLMs toward reliable oracles without encoding faulty behavior as intended.

The third direction designs recovery workflows that use evidence explicitly. Rather than one-shot generation, recovery can propose behavioral claims, check them against evidence, revise unsupported ones, and mark unresolved behavior as uncertain, via multi-sample consistency, critique-and-revision, and role-separated agents that propose, review, and refine specifications. Together, these target a workflow that recovers specifications when evidence suffices and propagates uncertainty when it does not.

4 Evaluation Methodology

The evaluation uses real-bug benchmarks with a buggy and a fixed version per defect, primarily Defects4J [13] and, where possible, BugsInPy [21]. We benchmark recovery in the test-prefix, test-oracle, and full test generation settings [3, 22], which isolate whether recovered intent yields bug-triggering inputs, intent-aligned rather than fault-aligned oracles, or complete effective tests. Baselines: direct code-based generation, generation with external documentation, non-LLM prefix generators such as EvoSuite [6], and our original single-prompt recovery, isolating the value of improved recovery.

Bug detection is the primary criterion, measured differentially: running each test on both versions distinguishes *effective tests* that detect the bug from *misguided* ones that assert the faulty behavior [23]. Every direction is judged by this criterion, its effect on effective and misguided test counts relative to the single-prompt baseline across models, so improvements in recovery are measured by downstream bug-detection impact. A direction succeeds only if it raises bug detection beyond the run-to-run noise band estimated from repeated trials; smaller effects count as noise unless replicated. Conditions differ only in the factor under study: evidence tiers are added one at a time to isolate each tier's marginal effect, representations are generated from the same evidence to attribute differences to form alone, and prompt size is reported to weigh gains against context cost. Because generated tests must also serve regression testing, each approach is further evaluated on the fixed version, where tests must pass without false alarms while maintaining coverage

and mutation score [4], a bar our prior approach already meets. To verify that recovery articulates intended behavior rather than merely raising detection rates, two annotators will independently label whether each recovered specification inherits the defect and whether it recovers the intended behavior, checked against developer-written documentation and fixed-version behavior, with Cohen's κ agreement and disagreements resolved by discussion. For the workflow direction, we also test whether the claims it flags as uncertain are those annotators mark as defect-inherited or unsupported, i.e., whether its uncertainty is calibrated. A key threat is data leakage, since LLMs may have seen benchmark projects and tests; we mitigate it by controlling model versions and cutoffs, adding post-cutoff bugs from GitBug-Java [19], and applying the approach to current project versions to reveal unknown bugs.

References

- [1] Azat Abdullin, Pouria Derakhshanfar, and Annibale Panichella. 2025. Test Wars: A Comparative Study of SBST, Symbolic Execution, and LLM-Based Approaches to Unit Test Generation. arXiv:2501.10200 [cs.SE] doi:10.48550/arXiv.2501.10200
- [2] Kent Beck. 2002. *Test-Driven Development: By Example*. Addison-Wesley Professional.
- [3] Yinghao Chen, Zehao Hu, Chen Zhi, Junxiao Han, Shuiguang Deng, and Jianwei Yin. 2024. ChatUniTest: A Framework for LLM-Based Test Generation. In *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering* (Porto de Galinhas, Brazil) (FSE 2024). Association for Computing Machinery, New York, NY, USA, 572–576. doi:10.1145/3663529.3663801
- [4] Arghavan Moradi Dakhel, Amin Nikanjam, Vahid Majdinasab, Foutse Khomh, and Michel C. Desmarais. 2024. Effective test generation using pre-trained Large Language Models and mutation testing. *Information and Software Technology* 171 (2024), 107468. doi:10.1016/j.infsof.2024.107468
- [5] Valentin Dallmeier, Nikolai Knopp, Andrzej Wasylkowski, and Andreas Zeller. 2010. Automatically Generating Test Cases for Specification Mining. In *Proceedings of the 19th International Symposium on Software Testing and Analysis (ISSTA '10)*. Association for Computing Machinery, New York, NY, USA, 85–96. doi:10.1145/1831708.1831719
- [6] Gordon Fraser and Andrea Arcuri. 2014. A Large-Scale Evaluation of Automated Unit Test Generation Using EvoSuite. *ACM Trans. Softw. Eng. Methodol.* 24, 2, Article 8 (Dec. 2014), 42 pages. doi:10.1145/2685612
- [7] Jerry Gao, H.-S. Jacob Tsao, and Ye Wu. 2003. *Testing and Quality Assurance for Component-Based Software*. Artech House.
- [8] Lehan He, Zeren Chen, Zhe Zhang, Jing Shao, Xiang Gao, and Lu Sheng. 2025. Use Property-Based Testing to Bridge LLM Code Generation and Validation. arXiv:2506.18315 [cs.SE] doi:10.48550/arXiv.2506.18315
- [9] Soneya Binta Hossain and Matthew B. Dwyer. 2025. TOGL: Correct and Strong Test Oracle Generation with LLMs. In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. 1475–1487. doi:10.1109/ICSE55347.2025.00098
- [10] Soneya Binta Hossain, Raygan Taylor, and Matthew Dwyer. 2025. Doc2OracLL: Investigating the Impact of Documentation on LLM-Based Test Oracle Generation. *Proc. ACM Softw. Eng.* 2, FSE, Article FSE084 (June 2025), 22 pages. doi:10.1145/3729354
- [11] Dong Huang, Jie M. Zhang, Mark Harman, Mingzhe Du, and Heming Cui. 2025. Measuring the Influence of Incorrect Code on Test Generation. arXiv:2409.09464 [cs.SE] doi:10.48550/arXiv.2409.09464
- [12] Laura Inozemtseva and Reid Holmes. 2014. Coverage is not strongly correlated with test suite effectiveness. In *Proceedings of the 36th International Conference on Software Engineering (Hyderabad, India) (ICSE 2014)*. Association for Computing Machinery, New York, NY, USA, 435–445. doi:10.1145/2568225.2568271
- [13] René Just, Darioush Jalali, and Michael D. Ernst. 2014. Defects4J: a database of existing faults to enable controlled testing studies for Java programs. In *Proceedings of the 2014 International Symposium on Software Testing and Analysis* (San Jose, CA, USA) (ISSTA 2014). Association for Computing Machinery, New York, NY, USA, 437–440. doi:10.1145/2610384.2628055
- [14] Tsz-On Li, Wenxi Zong, Yibo Wang, Haoye Tian, Ying Wang, Shing-Chi Cheung, and Jeff Kramer. 2023. Nuances are the Key: Unlocking ChatGPT to Find Failure-Inducing Tests with Differential Prompting. arXiv:2304.11686 [cs.SE] doi:10.48550/arXiv.2304.11686
- [15] Noble Saji Mathews and Meiyappan Nagappan. 2024. Design choices made by LLM-based test generators prevent them from finding bugs. arXiv:2412.14137 [cs.SE] doi:10.48550/arXiv.2412.14137
- [16] Michael Pradel and Thomas R. Gross. 2012. Leveraging Test Generation and Specification Mining for Automated Bug Detection without False Positives. In *Proceedings of the 34th International Conference on Software Engineering (ICSE '12)*. IEEE Press, 288–298. doi:10.1109/ICSE.2012.6227185
- [17] Corina S. Păsăreanu, Peter C. Mehlitz, David H. Bushnell, Karen Gundy-Burlet, Michael Lowry, Suzette Person, and Mark Pape. 2008. Combining unit-level symbolic execution and system-level concrete execution for testing NASA software. In *Proceedings of the 2008 International Symposium on Software Testing and Analysis* (Seattle, WA, USA) (ISSTA '08). Association for Computing Machinery, New York, NY, USA, 15–26. doi:10.1145/1390630.1390635
- [18] Sina Shamshiri, René Just, José Miguel Rojas, Gordon Fraser, Phil McMinn, and Andrea Arcuri. 2015. Do Automatically Generated Unit Tests Find Real Faults? An Empirical Study of Effectiveness and Challenges (T). In *2015 30th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 201–211. doi:10.1109/ASE.2015.86
- [19] André Silva, Nuno Saavedra, and Martin Monperrus. 2024. GitBug-Java: A Reproducible Benchmark of Recent Java Bugs. In *2024 IEEE/ACM 21st International Conference on Mining Software Repositories (MSR)*. IEEE Computer Society, Los Alamitos, CA, USA, 118–122. doi:10.1145/3643991.3644884
- [20] Junjie Wang, Yuchao Huang, Chunyang Chen, Zhe Liu, Song Wang, and Qing Wang. 2024. Software Testing With Large Language Models: Survey, Landscape, and Vision. *IEEE Transactions on Software Engineering* 50, 4 (April 2024), 911–936. doi:10.1109/TSE.2024.3368208
- [21] Ratnadira Widyasari, Sheng Qin Sim, Camellia Lok, Haodi Qi, Jack Phan, Qijin Tay, Constance Tan, Fiona Wee, Jodie Ethelda Tan, Yuheng Yieh, Brian Goh, Ferdian Thung, Hong Jin Kang, Thong Hoang, David Lo, and Eng Lieh Ouh. 2020. BugsInPy: a database of existing bugs in Python programs to enable controlled testing and debugging studies. In *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering* (Virtual Event, USA) (ESEC/FSE 2020). Association for Computing Machinery, New York, NY, USA, 1556–1560. doi:10.1145/3368089.3417943
- [22] Zhiqiang Yuan, Mingwei Liu, Shiji Ding, Kaixin Wang, Yixuan Chen, Xin Peng, and Yiling Lou. 2024. Evaluating and Improving ChatGPT for Unit Test Generation. *Proc. ACM Softw. Eng.* 1, FSE, Article 76 (July 2024), 24 pages. doi:10.1145/3660783
- [23] Junda Zhao, Shurui Zhou, and Eldan Cohen. 2026. Evaluating and Mitigating the Misguidance Effect of Buggy Code in LLM-Generated Unit Tests. *Proceedings of the ACM on Software Engineering* 3, ISSTA, Article ISSTA113 (Oct. 2026), 24 pages. doi:10.1145/3832204

Received 2026-07-04; accepted 2026-08-04